



Модель DOM

Преподаватель:

канд. техн. наук, доц. Озерова Г.П.

Document Object Model

Основным инструментом для динамических изменений на HTML-странице является DOM (Document Object Model).

DOM используется для того, чтобы манипулировать страницей – читать информацию из HTML, создавать и изменять элементы страницы.

Дерево документа

Согласно DOM-модели, документ можно представить в виде дерева.

Каждый HTML-тег образует узел дерева с типом **элемент**.

Вложенные в него теги становятся дочерними узлами.

Для представления текста создаются узлы типа **текст**.

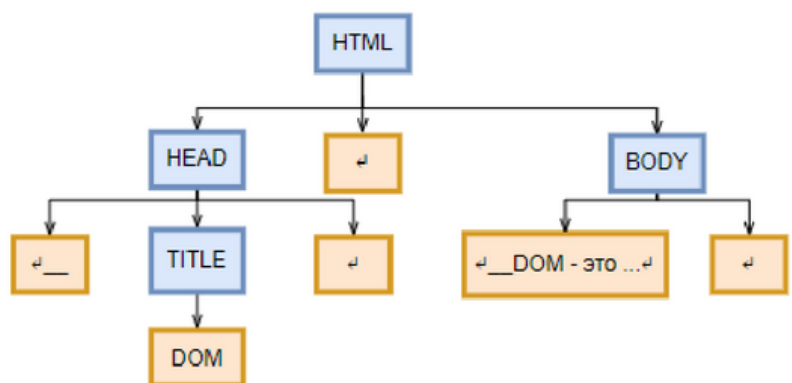
Это описание документа доступно для изменения через JavaScript.

Дерево документа

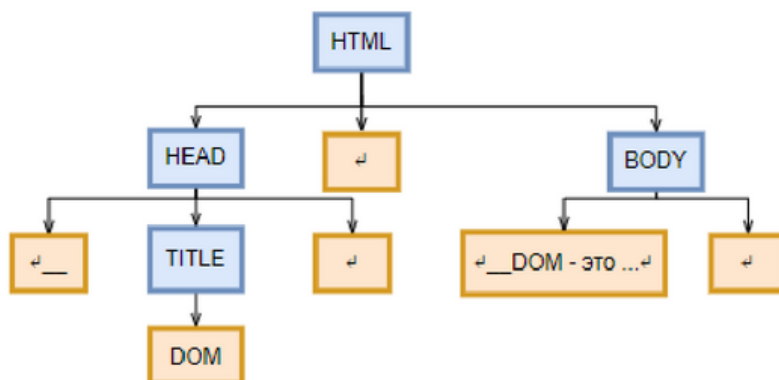
HTML-код:

```
<html>
<head>
  <title>DOM</title>
</head>
<body>
  DOM - это ...
</body>
</html>
```

Дерево:



Дерево документа



В этом дереве выделено два типа узлов.

- Узлы-элементы (обозначены синим цветом) образуются тегами HTML. В соответствии с HTML-кодом, одни узлы вложены в другие, структура дерева создается за счет них.
- Текст внутри элементов образует текстовые узлы (обозначены желтым цветом). Текстовый узел содержит исключительно строку текста и не может иметь потомков. Пробелы (_) и переводы строки (↵) – это тоже текст, который учитываются в DOM.

Дерево документа

При включении текстовых элементов в дерево необходимо учитывать следующее:

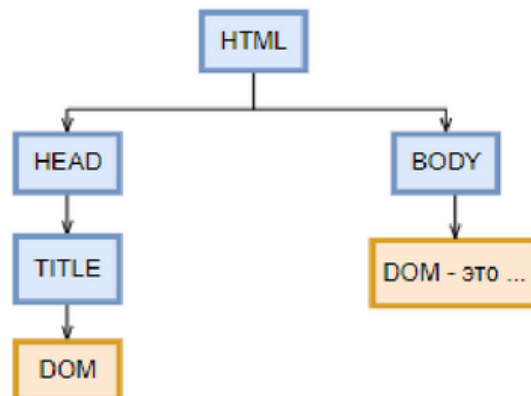
- пробелы и переносы строк до **<head>** игнорируются;
- любое содержимое после **</body>** не создает узла, браузер переносит его в конец **body**.

Дерево документа

HTML-код:

```
<html><head><title>DOM</title></head><body>DOM -  
...</body></html>
```

Дерево:



Дерево документа

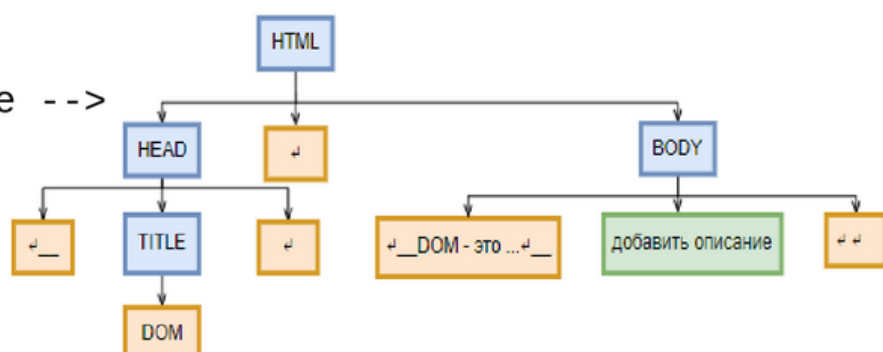
Кроме узлов-элементов и текстовых узлов в дерево включаются узлы-комментарии.

HTML-код:

```
<html>  
<head>  
  <title>DOM</title>  
</head>  
<body>
```

```
  DOM - это...  
  <!-- добавить описание -->  
</body>  
</html>
```

Дерево:



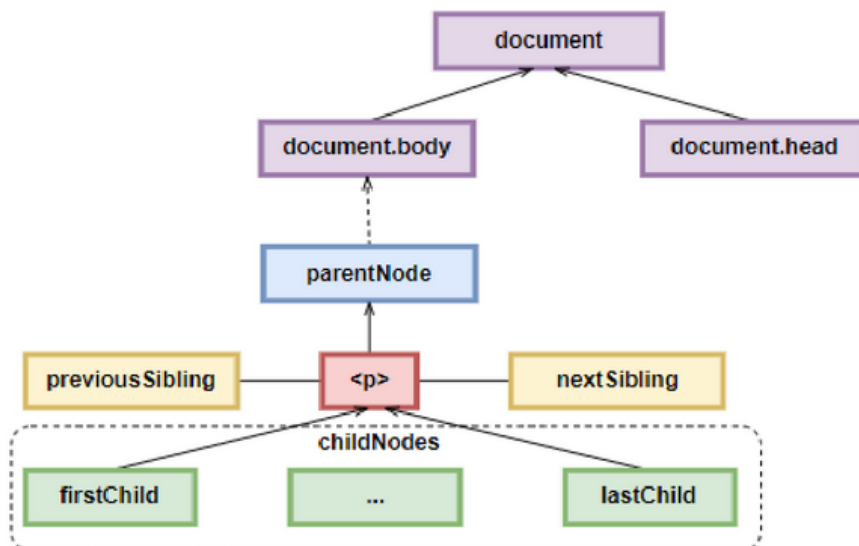
Навигация по элементам

Особенности построения дерева документа

1. В дереве всегда будут теги **<html>**, **<head>** и **<body>**, даже если в HTML-документе их нет.
2. При чтении ошибочного HTML браузер автоматически корректирует его при построении: незакрытые теги будут закрыты и т.д.
3. Таблицы в дереве документа всегда содержат тег **<tbody>**, хотя при написании HTML-кода этот тег обычно пропускается.

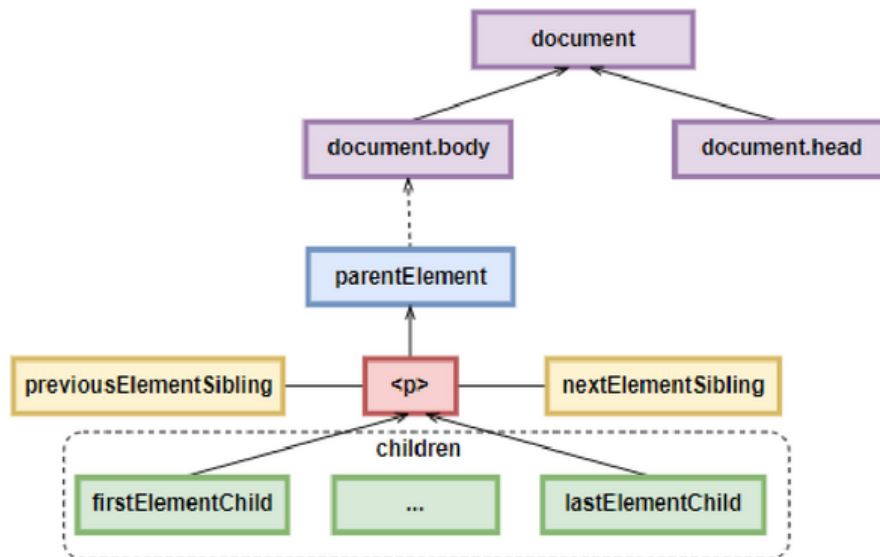
Навигация по элементам

Ссылки для перехода по всем узлам дерева



Навигация по элементам

Ссылки для перехода по узлам-элементам



Навигация по элементам

Важно учитывать, что **нельзя получить доступ к элементу, которого еще не существует в момент выполнения скрипта.**

```
<html>
<head>
  <script>
    // выведет: Из HEAD - null
    alert( "Из HEAD - " + document.body );
  </script>
</head>
<body>
  <script>
    //выведет: Из BODY - [objectHTMLBodyElement]
    alert( "Из BODY - " + document.body );
  </script>
</body>
</html>
```

Коллекции элементов

Для хранения некоторой совокупности элементов дерева DOM используется **коллекции**. Коллекции в чем-то похожи на массивы:

- обращение к элементам коллекции осуществляется по индексу;
- количество элементов определяется свойством **length**.

Но к коллекциям не применимы многие методы массивов, например, такие как **forEach()**, **push()**, **pop()**, также у них есть свои свойства и методы, которых в массивах нет.

По-другому коллекции называют **псевдомассивами**.

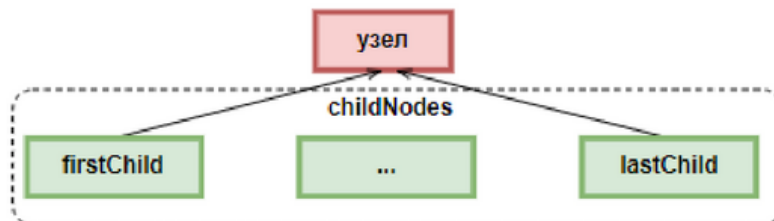
Дочерние узлы и потомки

Дочерние узлы (или дети) – элементы, которые лежат непосредственно внутри некоторого элемента. Например, для **<html>** это **<head>** и **<body>**.

Термин **потомки** применяется для описания всего поддерева некоторого элемента (то есть всех элементов, лежащих внутри данного вместе с их детьми, детьми их детей т.д.).

Дочерние узлы и потомки

Коллекция `childNodes`



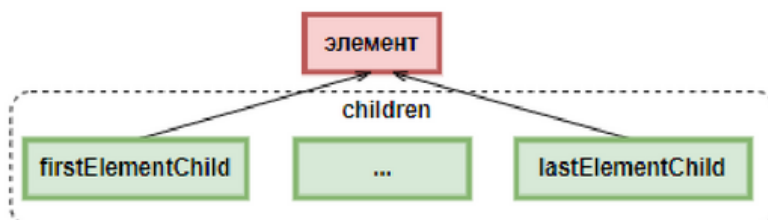
Коллекция **`childNodes`** хранит все дочерние узлы некоторого элемента, включая текстовые и комментарии.

Эта коллекция предназначена только для чтения элементов.

Свойства **`firstChild`** и **`lastChild`** используются для доступа к первому и последнему дочернему элементу.

Дочерние узлы и потомки

Коллекция `children`



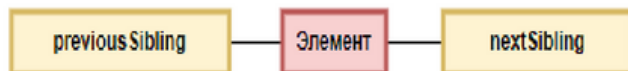
Для того чтобы осуществлять навигацию только по узлам-элементам используются специальные свойства:

- **`children`** – коллекция из дочерних узлов-элементов.
- **`firstElementChild`**, **`lastElementChild`** – первый и последний дочерний элемент.

Коллекцию **`children`** можно корректировать.

Соседи

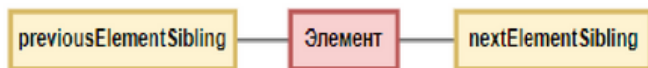
Доступ к узлам любого типа слева и справа от данного можно получить по ссылкам:



- **previousSibling;**
- **nextSibling**

Соседи

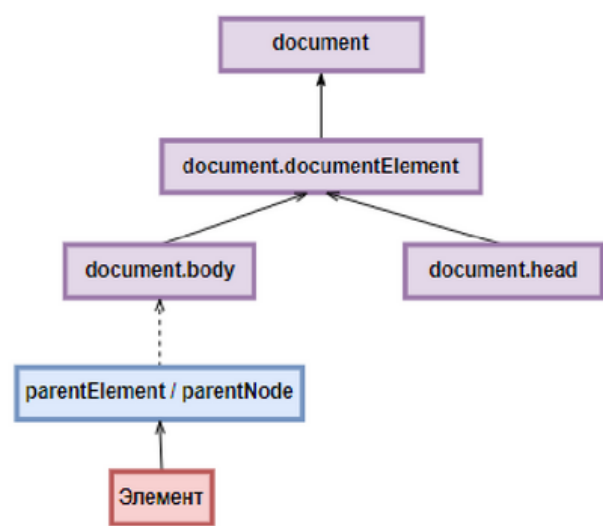
Навигация по узлам-элементам осуществляется с помощью ссылок:



- **previousElementSibling**
- **nextElementSibling.**

Если правого или левого соседа у элемента нет - соответствующее свойство имеет значение **null**.

Родитель



Родитель узла любого типа доступен через ссылку **parentNode**.

Ссылка **parentElement** – родитель-элемент, эквивалентен **parentNode** за исключением обращения к родителю **documentElement**:

```
alert( document.documentElement.parentNode ); // document
```

```
alert( document.documentElement.parentElement ); // null
```

Поиск элементов в дереве

Метод	Ищет по...	Формирует коллекцию?	Ищет внутри элемента?
getElementById	id	-	-
getElementByName	name	-	-
getElementsByTagName	тегу или *	✓	✓
getElementsByClassName	классу	✓	✓

Поиск элементов в дереве

Если **id** элемента отвечает требованиям к выбору имен переменных в JavaScript, то к элементу можно обратиться непосредственно по **id** без использования метода `getElementById()`.

```
document.getElementById('menu');
```

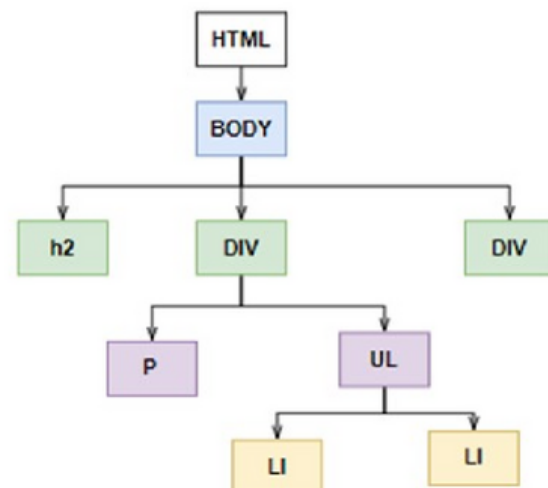
эквивалентно

```
menu
```

Поиск элементов в дереве

Пример. Рассмотрим html-документа и его дерево элементов.

```
<html>
  <body>
    <h2 class="bold"> Заголовок </h2>
    <div id = menu>
      <p class="bold">Оглавление</p>
      <ul>
        <li>Глава 1</li>
        <li class="bold">Глава 2</li>
      </ul>
    </div>
    <div class="bold"> Примечания </div>
  </body>
  <script src="program.js"></script>
</html>
```



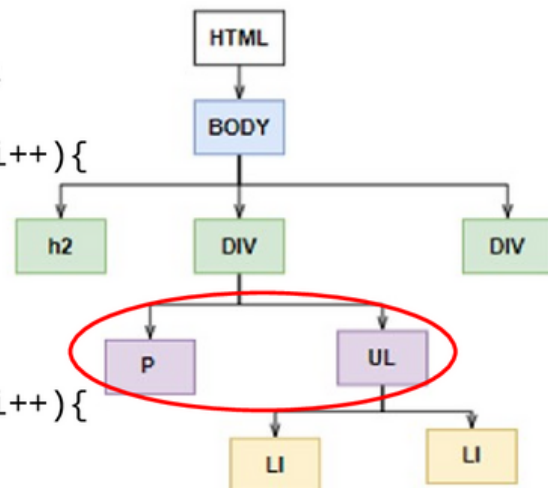
Поиск элементов в дереве

Пример. Найдем всех детей элемента с **id menu**.

```
let item = document.getElementById('menu');  
  
for (let i = 0; i < item.children.length; i++){  
    alert(`${ item.children[i] }`);  
}
```

либо

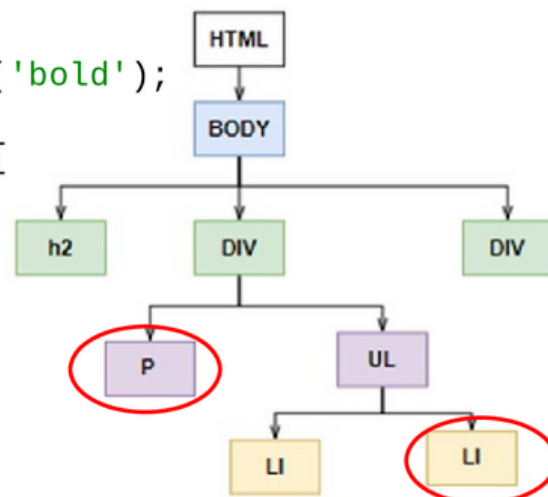
```
for (let i = 0; i < item.children.length; i++){  
    alert(`${ menu.children[i] }`);  
}
```



Поиск элементов в дереве

Пример. Найдем элементы, относящиеся к классу **bold**, являющиеся потомками элемента с **id menu**.

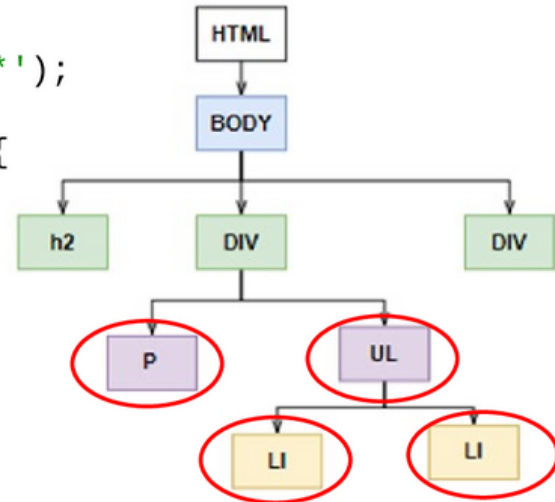
```
let elements = menu.getElementsByTagName('bold');  
  
for (let i = 0; i < elements.length; i++) {  
    alert(elements[i]);  
}
```



Поиск элементов в дереве

Пример. Найдем элементы, являющиеся потомками элемента с `id menu`.

```
let elements = menu.getElementsByTagName('*');  
for (let i = 0; i < elements.length; i++) {  
  alert(elements[i]);  
}
```



Поиск элементов в дереве

Задание 1. Вывести все элементы страницы, которые имеют ровно 3 потомка.

Свойства узлов дерева DOM

Свойство	Описание	Значения	Доступно для изменения?
nodeType	Содержит тип узла.	ELEMENT_NODE (1) ATTRIBUTE_NODE (2) TEXT_NODE (3)	-
nodeName	Содержит название тега (большими буквами).	BODY, #text , #comment	-
className	Содержит название класса.		✓
style	Содержит описание притеговых стилей элемента.		✓
innerHTML	Содержит содержимое HTML документа.		✓
outerHTML	Содержит HTML элемента целиком.		-
textContent	Содержит текст внутри элемента без тегов.		✓ (все вложенные теги теряются)
hidden	Управляет видимостью элемента.	true - невидимый элемент false - видимый элемент	✓

Свойства узлов дерева DOM

Пример. Изменим существующий список в элементе **div** с **id menu**, дописав к строкам нумерованного списка текстовые строки из массива.

HTML-код:

```
<html>
  <body>
    <h2 class="bold"> Заголовок </h2>
    <div id = "menu">
      <p class="bold">Оглавление</p>
      <ul>
        <li>Глава 1</li>
        <li>Глава 2</li>
      </ul>
    </div>
    <div class="bold"> Примечания </div>
  </body>
  <script src="program.js"></script>
</html>
```

Свойства узлов дерева DOM

Пример. Изменим существующий список в элементе **div** с **id menu**, дописав к строкам нумерованного списка текстовые строки из массива.

JavaScript-код:

```
let arrText = ['Дерево DOM', 'Навигация по DOM'];

let elem = document.getElementById('menu');

let elementsLi = elem.getElementsByTagName('li');

for (let i = 0; i < elementsLi.length; i++) {
  elementsLi[i].innerHTML += ` ${ arrText[i] }`;
}
```

Свойства узлов дерева DOM

Задание 2. У всех списков **ol** скрыть все четные дочерние элементы

Свойства созданные пользователем

Узел дерева DOM – это объект, поэтому, как и любой объект в JavaScript, он может содержать пользовательские свойства и методы.

Эти свойства и методы видны только в JavaScript и никак не влияют на отображение соответствующего тега.

Свойства, созданные пользователем

Пример. Создадим в `document.body` новое свойство и метод

JavaScript-код:

```
document.body.message = "Привет, мир!";

document.body.showMessage = function() {
    alert(`Значение нового свойства: ${this.message} \n
        Teg: ${this.nodeName}`);
}

document.body.showMessage();
```

Свойства, созданные пользователем

Пример. Создадим в `document.body` новое свойство и метод

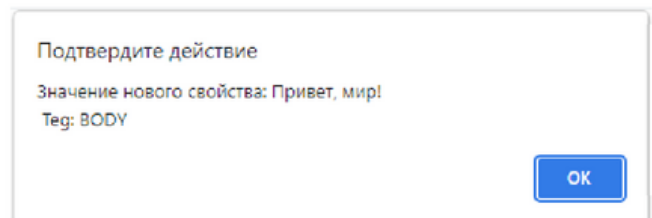
JavaScript-код:

```
document.body.message = "Привет, мир!";

document.body.showMessage = function() {
    alert(`Значение нового свойства: ${this.message} \n
        Teg: ${this.nodeName}`);
}

document.body.showMessage();
```

Результат:



DOM - атрибуты тегов

Пример. Для элемента с заданным `id` установим атрибут "выравнивание по центру", а затем выведем все атрибуты этого элемента.

HTML-код:

```
<body>
    <h2 id = "title" class="bold"> Заголовок </h2>
    ...
</body>
```

JavaScript-код:

```
let item = document.getElementById('title');
item.setAttribute('align', 'center');
let attrs = item.attributes;
let str = '';
for (let i = 0; i < attrs.length; i++) {
    str += `${attrs[i].name} = ${attrs[i].value}\n`;
}
alert(str);
```

DOM - атрибуты тегов

Пример. Для элемента с заданным **id** установим атрибут "**выравнивание по центру**", а затем выведем все атрибуты этого элемента.

HTML-код:

```
<body>
  <h2 id = "title" class="bold"> Заголовок </h2>
  ...
</body>
```

JavaScript-код:

```
let item = document.getElementById('title');
item.setAttribute('align', 'center');
let attrs = item.attributes;
let str = '';
for (let i = 0; i < attrs.length;
    str += `${ attrs[i].name } = ${`
  alert(str);
```

Подтвердите действие

id = title
class = bold
align = center

OK

DOM - атрибуты тегов

Задание 3. Во всех элементах на странице удалить атрибуты **align**, **width** и **height**.

DOM - классы CSS

Свойство/метод	Описание	Доступно для изменения?
<code>className</code>	Позволяет получить/задать класс элементу (соответствует атрибуту <code>class</code>).	✓
<code>classList.contains(class)</code>	Возвращает true/false , в зависимости от того, есть ли у элемента класс с именем class .	-
<code>classList.add(class)</code>	Добавляет класс с именем class , если такого в списке классов нет	✓
<code>classList.remove(class)</code>	Удаляет класс с именем class .	✓
<code>classList.toggle(class)</code>	Если класса с именем class нет, добавляет его, если есть - удаляет.	✓
<code>classList</code>	Коллекция классов элемента.	✓

DOM - классы CSS

Пример. Для всех элементов, относящихся к тегу `<p>`, добавить класс **bold**, если у элемента этого класса нет.

HTML-код:

```
<body>
  <h2 class="bold"> Заголовок </h2>
  <div>
    <p class="bold">Оглавление</p>
    <ul>
      <li>Глава 1</li>
      <li>Глава 2</li>
    </ul>
  </div>
  <div class="bold">Примечания</div>
  <p>Приложения</p>
</body>
<script src="program.js"></script>
```


DOM - классы CSS

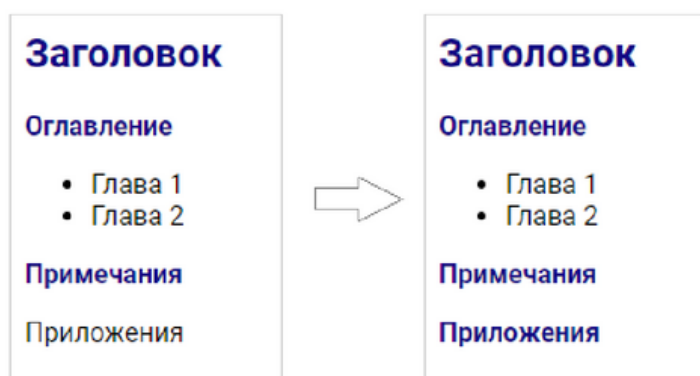
Пример. Для всех элементов, относящихся к тегу `<p>`, добавить класс **bold**, если у элемента этого класса нет.

JavaScript-код:

```
let items = document.getElementsByTagName('p');

for (let i = 0; i < items.length; i++) {
    items[i].classList.add('bold');
}
```

Результат :



DOM - классы CSS

Задание 4. У всех рисунков, имеющих атрибуты **width** и **height**, удалить этот атрибут и добавить класс **pict**.

Создание и вставка элементов

Свойство	Описание
<code>document.createElement(tag)</code>	Создает новый элемент с указанным тегом.
<code>document.createTextNode(text)</code>	Создает новый текстовый узел с указанным текстом.
<code>node.cloneNode(true/false)</code>	Создает копию node , если указан параметр true , то копия создается вместе с атрибутами, включая подэлементы. С параметром false копия будет сделана без дочерних элементов.
<code>parent.appendChild(node)</code>	Добавляет элемент node в конец дочерних элемента parent .
<code>parent.insertBefore(node, nextSibling)</code>	Вставляет node в коллекцию детей элемента parent , перед элементом nextSibling .

Вставка элементов

Свойство	Описание
<code>node.append(nodes или strings)</code>	Добавляет узлы или строки в конец node
<code>node.prepend(nodes или strings)</code>	Добавляет узлы или строки в начало node
<code>node.before(nodes или strings)</code>	Вставляет узлы или строки до node
<code>node.after(nodes или strings)</code>	Вставляет узлы или строки после node
<code>node.replaceWith(nodes или strings)</code>	Заменяет node заданными узлами или строками

Создание и вставка элементов

Пример. Применим методы создания элементов и вставки для следующего HTML-кода:

```
<ul id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ul>
```

Создание и вставка элементов

JavaScript-код:

```
list.before('before'); // вставить строку "before" перед <ul>
list.after('after'); // вставить строку "after" после <ul>
```

HTML-код:

```
before
<ul id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ul>
after
```

Создание и вставка элементов

JavaScript-код:

```
let liFirst = document.createElement('li');
liFirst.innerHTML = 'prepend';
list.prepend(liFirst); // вставить liFirst в начало <ul>
```

HTML-код:

```
before
<ul id="list">
  <li>prepend</li>
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ul>
after
```

Создание и вставка элементов

JavaScript-код:

```
let liLast = document.createElement('li');
liLast.innerHTML = 'append';
list.append(liLast); // вставить liLast в конец <ul>
```

HTML-код:

```
before
<ul id="list">
  <li>prepend</li>
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>append</li>
</ul>
after
```

Создание и вставка элементов

JavaScript-код:

```
let liReplace = document.createElement('li');
liReplace.innerHTML = 'replace';
list.children[2].replaceWith(liReplace); // заменить второго
ребенка <ul>
```

HTML-код:

before

```
<ul id="list">
  <li>prepend</li>
  <li>0</li>
  <li>replace</li>
  <li>2</li>
  <li>append</li>
</ul>
```

after

Вставка элементов

Пример. Поменяем местами первый и последний элемент **section**.

HTML-код:

```
<article>
  <section>HTML</section>
  <section>CSS</section>
  <section>JavaScript</section>
</article>
```

HTML

CSS

JavaScript

Вставка элементов

Пример. Поменяем местами первый и последний элемент `section`.

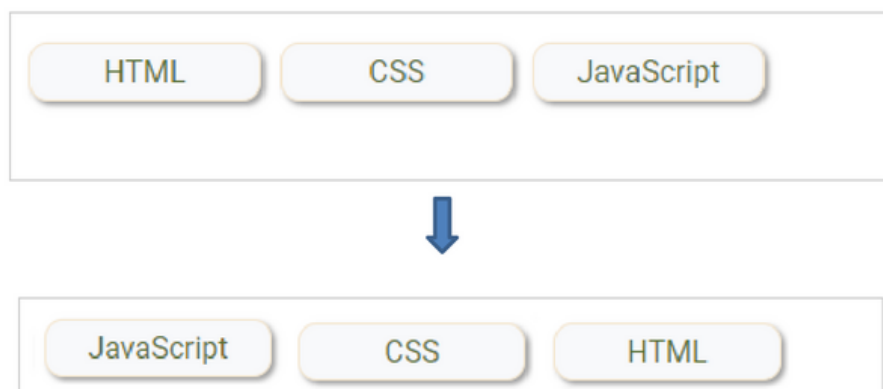
JavaScript-код:

```
let article = document.getElementsByTagName('article')[0];  
  
//последний элемент добавляем перед первым  
article.prepend(article.lastElementChild);  
  
//старый первый (новый второй) - добавляем в конец  
article.append(article.children[1]);
```

Вставка элементов

Пример. Поменяем местами первый и последний элемент `section`.

Результат:



Вставка HTML-кода

Для вставки в документ HTML-кода используется метод:

node.insertAdjacentHTML(where, html)

where – это специальное слово, указывающее, куда по отношению к **node** производить вставку. Возможные значения:

"beforebegin" – вставить **html** непосредственно перед **node**,

"afterbegin" – вставить **html** в начало **node**,

"beforeend" – вставить **html** в конец **node**,

"afterend" – вставить **html** непосредственно после **node**.

html – это HTML-строка, которая будет вставлена «как HTML».

Создание и вставка элементов

Пример. Применим метод **insertAdjacentHTML** для следующего HTML-кода:

```
<ul id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ul>
```

Создание и вставка элементов

JavaScript-код:

```
list.insertAdjacentHTML('beforebegin', '<p>before</p>');
list.insertAdjacentHTML('afterend', '<p>after</p>');
list.insertAdjacentHTML('afterbegin', '<li>prepend</li>');
list.insertAdjacentHTML('beforeend', '<li>append</li>');
```

HTML-код:

```
<p>before</p>
<ul id="list">
  <li>prepend</li>
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>append</li>
</ul>
<p>after</p>
```

Создание и вставка элементов

Задание 5. Создать абзацы с текстом на основе массива **paragraphs** и поместить их в блок с **id text** следующим образом:

<pre><div id="text"> <p>Python</p> <p>SQL</p> <p>PHP</p> </div></pre>		<pre><div id="text"> <p>C++</p> <p>Python</p> <p>C#</p> <p>SQL</p> <p>JavaScript</p> <p>PHP</p> </div></pre>
---	---	--

```
paragraphs = ['C++', 'C#', 'JavaScript'];
```

Удаление элементов

Свойство	Описание
<code>parent.removeChild(node)</code>	Удаляет node из списка детей parent , а также возвращает удаленный элемент.
<code>parent.replaceChild(newNode, node)</code>	Среди детей parent удаляет node и вставляет на его место newNode , а также возвращает удаленный элемент.
<code>node.remove()</code>	Напрямую удаляет элемент node .

Удаление элементов

Пример. Дан список:

```
<ul id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
</ul>
```

Как он будет выглядеть после выполнения следующего скрипта:

```
let i = 0;
while (i < list.children.length) {
  list.children[i].remove();
  i++;
}
```

Удаление элементов

Пример. Дан список:

```
<ul id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
</ul>
```



```
<ul id="list">
  <li>1</li>
  <li>3</li>
  <li>5</li>
</ul>
```

Как он будет выглядеть после выполнения следующего скрипта:

```
let i = 0;
while (i < list.children.length) {
  list.children[i].remove();
  i++;
}
```

Удаление элементов

Задание 6. Очистить элемент с **id menu**. Свойство **innerHTML** не использовать.

Спасибо за внимание!